DAY 2 | HANDS-ON ACTIVITIES

Python Basics, Functions and Files: Problem Statements

Nine problems from warm-up to stretch. Solve them in the Day 2 notebook or in a .py file in VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Temperature Converter

Level: Warm-up | **Concepts:** variables, float(), arithmetic, f-strings

Store a temperature in Celsius and print it in Fahrenheit. Formula: F = C x 9 / 5 + 32. Round to 1 decimal place. Then make it read the value with input().

Sample run / expected result

```
36.6 C is 97.9 F
```

Hints

- input() returns text, so cast it with float().
- round(value, 1) rounds to one decimal place.

Problem 2: Grade Calculator

Level: Easy | **Concepts:** if/elif/else, comparison operators

Given marks out of 100, print the grade: 85 and above is A, 70 to 84 is B, 50 to 69 is C, below 50 is "Needs improvement". Marks below 0 or above 100 print "Invalid marks".

Sample run / expected result

```
marks = 78  ->  Grade: B
marks = 120 ->  Invalid marks
```

Hints

- Check the invalid range first.
- Order your elif checks from highest to lowest.

Problem 3: Number Guessing Game

Level: Medium | **Concepts:** while loop, break, conditions

Store a secret number between 1 and 20. Allow up to 5 guesses. Say "Too high" or "Too low" after each wrong guess; stop when correct. Test it with a fixed list of guesses first.

Sample run / expected result

```
Guess 10: Too low
Guess 15: Too high
Guess 13: Correct! You got it in 3 attempts.
```

Hints

- Keep an attempts counter.
- Use break when the guess is correct.

Problem 4: Area Helpers

Level: Easy | **Concepts:** def, parameters, return

Write rectangle_area(l, w), circle_area(r) and triangle_area(b, h). Call each and print the result rounded to 2 decimals.

Sample run / expected result

```
rectangle_area(4, 5) -> 20
circle_area(3)       -> 28.27
triangle_area(6, 4)  -> 12.0
```

Hints

- import math and use math.pi.

Problem 5: Word Counter

Level: Medium | **Concepts:** dict, split, functions, sorted

Write count_words(text) that returns a dict of each word and how often it appears (ignore case). Print the three most common words.

Sample run / expected result

```
count_words("the cat and the hat and the bat")
{'the': 3, 'cat': 1, 'and': 2, 'hat': 1, 'bat': 1}
Top 3: the, and, cat
```

Hints

- text.lower().split()
- sorted(d.items(), key=lambda kv: kv[1], reverse=True)

Problem 6: Student Report

Level: Medium | **Concepts:** list of dicts, functions

Store five students as a list of dicts with name and marks. Write average(students) and topper(students). Print the class average and the topper.

Sample run / expected result

```
Class average: 74.4
Topper: Priya (92)
```

Hints

- max(students, key=lambda s: s["marks"])

Problem 7: Expenses to CSV

Level: Challenging | **Concepts:** csv module, dicts

Write a list of expenses (date, category, amount) to expenses.csv. Read it back and print the total spent per category.

Sample run / expected result

```
Food: 1450.0
Travel: 800.0
Bills: 2200.0
```

Hints

- Values read from CSV are text: cast with float().
- Accumulate totals in a dict.

Problem 8: Contact Book with JSON

Level: Challenging | **Concepts:** functions, dict, json, files

Build add_contact, find_contact, save_contacts and load_contacts. Contacts are stored in contacts.json so they survive restarts.

Sample run / expected result

```
add_contact("Mei", "98450 11111")
save_contacts()
# restart
load_contacts()
find_contact("Mei") -> 98450 11111
```

Hints

- load returns an empty dict if the file does not exist yet.

Problem 9: SmartDesk Ticket Logger

Level: Stretch | **Concepts:** functions, list of dicts, json, CLI loop

SmartDesk step 1. Write load_tickets(), save_tickets(tickets), add_ticket(tickets, customer, issue, priority) and list_tickets(tickets, status=None). Tickets are saved to tickets.json. Add a menu: 1) Add 2) List 3) Exit. Reject priorities other than low, medium, high.

Sample run / expected result

```
1) Add 2) List 3) Exit > 1
Customer: Mei
Issue: No signal at home
Priority: high
Saved ticket #1
> 2
#1 [high] Mei: No signal at home (open)
```

Hints

- Each ticket is a dict with id, customer, issue, priority, status.
- Save after every change so nothing is lost.